FPT Education
FPT UNIVERSITY

Chapter 5: Kernel Initialization

From Bootloader → Kernel → init (PID 1)

FPT Education
FPT UNIVERSITY

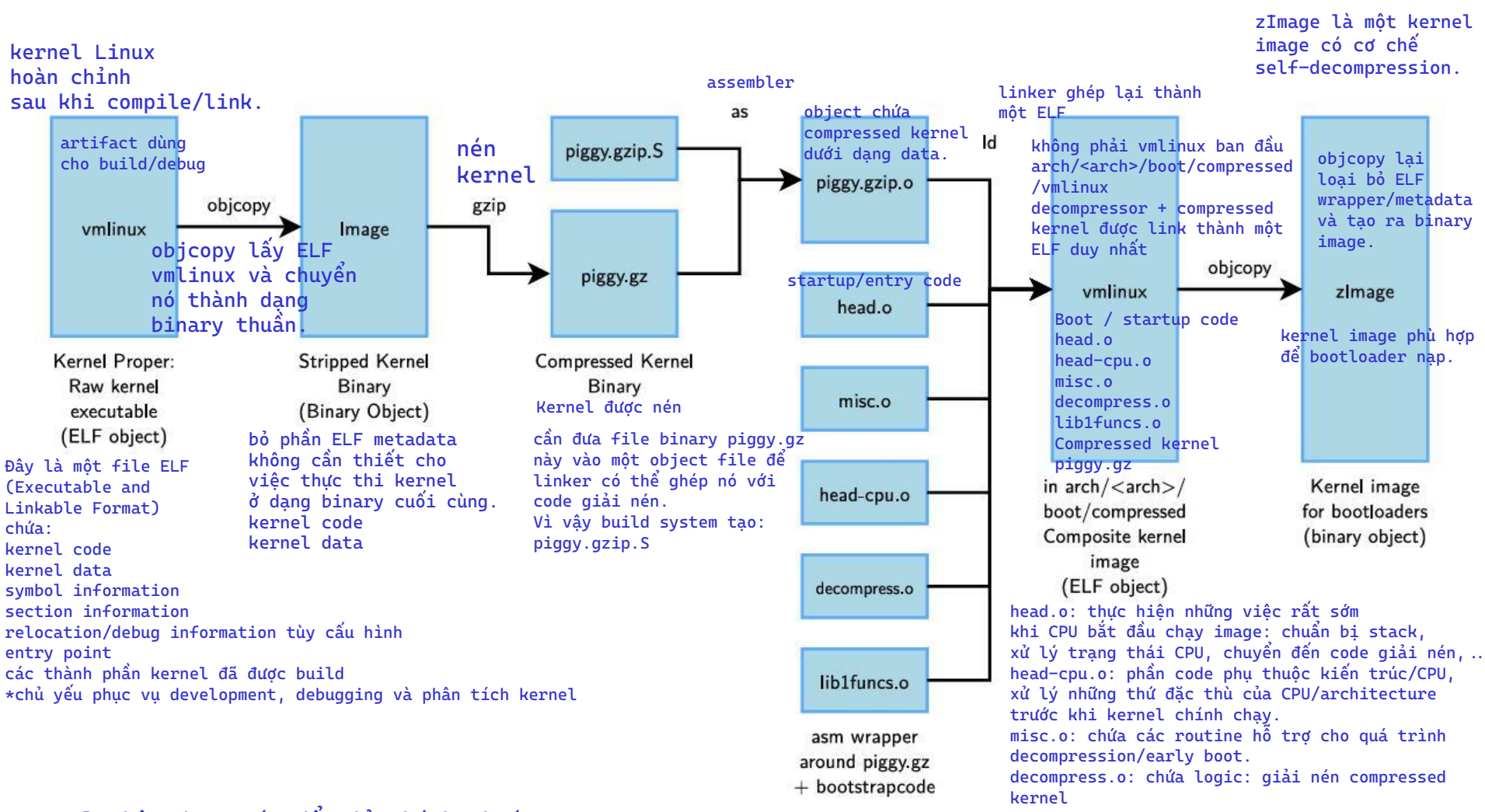
Learning Objectives

- Explain kernel initialization flow
- Understand entry point (head.o) and main.c
- Analyze initcall mechanism
- Understand init process (PID 1)

FPT Education

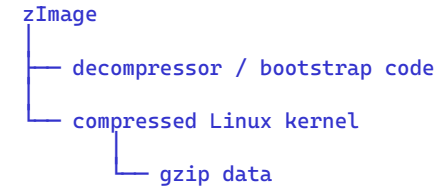
FPT UNIVERSITY

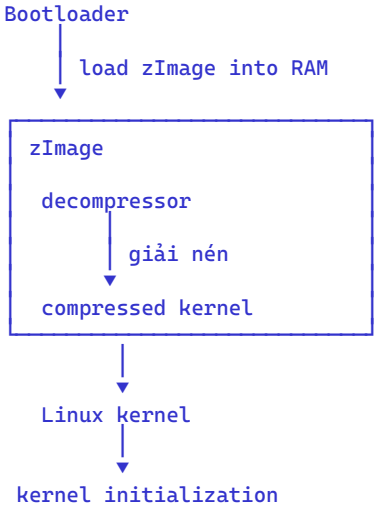
Kernel Initialization Overview

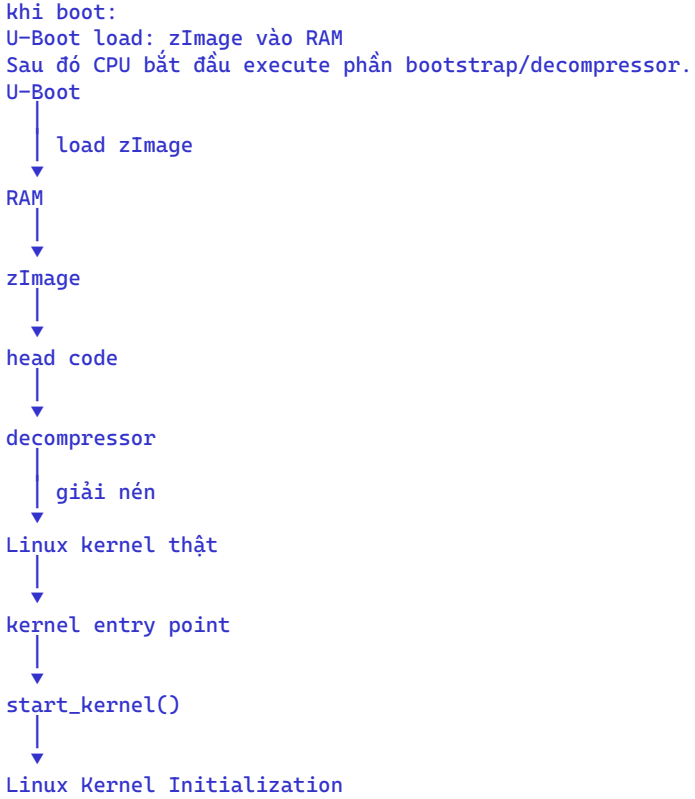


Kernel thật được nén để giảm kích thước.
Nhưng CPU/bootloader cần một đoạn code nhỏ để giải nén kernel trước khi chạy kernel thật. Vì vậy zImage chứa cả "decompressor + compressed kernel".

vmlinux chủ yếu là artifact dùng cho build/debug, không phải dạng kernel image tối ưu để bootloader trực tiếp nạp trên các kiến trúc sử dụng zImage.







Flow thực tế có thể chia thành 2 giai đoạn lớn:
Giai đoạn 1 — Kernel image preparation/bootstrapping
vmlinux
↓
image
↓
piggy.gz
↓
decompressor + piggy.gz
↓
zImage
(build-time)

Giai đoạn 2 — Runtime boot
Bootloader
↓
load zImage
↓
decompressor
↓
decompressed kernel
↓
architecture-specific startup
↓
start_kernel()
↓
Linux kernel initialization
↓
init process
↓
user space
(runtime)

File	Ý nghĩa
`vmlinux`	Kernel ELF đầy đủ sau build/link
`image`	Kernel binary chưa nén
`piggy.gz`	Kernel binary đã gzip
`arch/ … /compressed/vmlinux`	ELF chứa decompressor + compressed kernel
`zImage`	Binary cuối cùng để bootloader nạp

đơn giản hóa

vmlinux
↓ bỏ ELF metadata
image
↓ gzip
piggy.gz
↓ nhúng vào decompressor
compressed/vmlinux
↓ bỏ ELF metadata
zImage

zImage =
[decompressor]
+
[compressed kernel]

Lưu ý:
- Không phải mọi architecture đều tạo zImage; ví dụ ARM64 thường dùng Image, x86 có các dạng image khác.
- Tên và tập hợp các object như head.o, misc.o, head-cpu.o, lib1funcs.o phụ thuộc architecture và phiên bản kernel.
- vmlinux trong root kernel tree và arch/<arch>/boot/compressed/vmlinux là hai artifact khác nhau.
- piggy.gzip.S về bản chất là assembly wrapper để đưa compressed kernel vào link image, chứ không phải một "kernel source file" thông thường.

Anatomy of the Kernel Image

- vmlinux: ELF kernel with debug symbols
- Image/zImage/bzImage: bootable kernel
  - Raspberry Pi 4/5 commonly uses ARM64 Image kernel format.
  - NXP i.MX8, TI AM62x, and Rockchip RK3588 platforms heavily use FIT images in modern embedded Linux deployments.
  - Modern U-Boot increasingly treats FIT (Flattened Image Tree) as the standard deployment format for secure embedded systems.
- piggy.gz: compressed kernel payload
- Compression reduces boot time and size

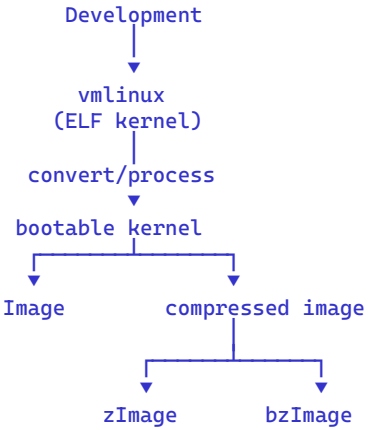
ARM64

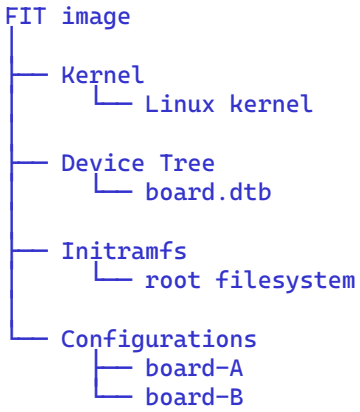
các dạng kernel image dùng trong boot

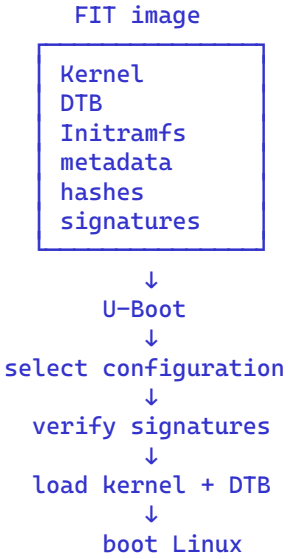
Flattened Image Tree

FIT thường dùng Flattened Image Tree / Device Tree syntax để mô tả và đóng gói nhiều thành phần boot thành một image.

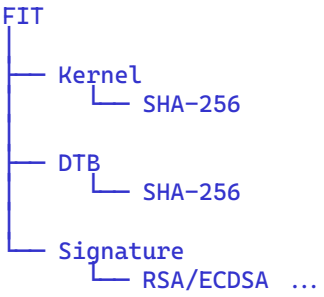
FIT có thể chứa hashes và digital signatures.

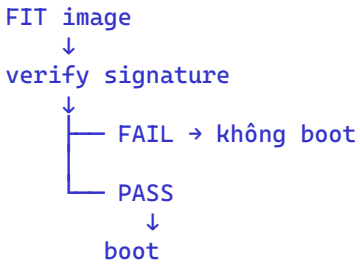




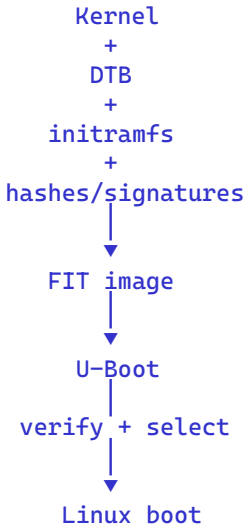


secure boot:





embedded linux hiện đại:



FPT Education
FPT UNIVERSITY

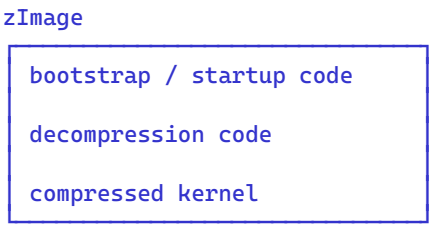
Bootloader vs Bootstrap Loader

- Bootloader (e.g., U-Boot): loads kernel
- Bootstrap: Một đoạn code khởi động ban đầu có nhiệm vụ chuẩn bị mọi thứ cần thiết để Linux kernel có thể bắt đầu chạy đúng cách.
  - decompresses: zImage (ARM32), bzImage (x86)
  - On AArch64, kernel Image don't decompression. ARM64 Image là dạng kernel image không nén; nếu muốn compression, boot chain/platform có thể sử dụng một cơ chế khác.
  - Prepares kernel

Boot ROM là code nằm trong ROM/boot memory của chip, do nhà sản xuất SoC cung cấp.

- Boot ROM → Bootloader → Kernel
- Bootstrap often embedded in kernel

Bootstrap/early boot code thường được build cùng kernel và nằm trong bootable kernel image.



zImage

```
bootstrap code

decompression code

compressed kernel
    (4 MB)
```

```
Bootloader
    ↓
load zImage
    ↓
RAM
    ↓
execute bootstrap
    ↓
decompress
    ↓
10 MB kernel
    ↓
jump to kernel
```

zimage:

```
CPU
 ↓
chạy bootstrap code
 ↓
thiết lập môi trường tối thiểu
 ↓
giải nén kernel (nếu kernel bị nén)
 ↓
nhảy đến kernel
```

Image:

```
Bootloader
    ↓
load Image
    ↓
RAM
    ↓
execute Image
    ↓
Linux kernel
```

prepare kernel:

```
Bootstrap
   ↓
initial CPU/memory setup
   ↓
kernel entry
   ↓
start_kernel()
   ↓
Linux kernel initialization
   ↓
drivers
scheduler
memory management
VFS
network
 ...
```

tầng 1

Boot ROM thường làm những việc tối thiểu như:
xác định boot source
đọc boot device
tìm bootloader
có thể thực hiện secure boot verification
load bootloader vào RAM
jump đến bootloader

tầng 2

U-Boot có nhiệm vụ phức tạp hơn Boot ROM:

```
U-Boot
 │
 ├── initialize hardware
 ├── initialize DRAM
 ├── read storage/network
 ├── load kernel ⇒ RAM
 ├── load DTB ⇒ RAM
 ├── load initramfs
 ├── verify FIT/signature
 └── jump to kernel
```

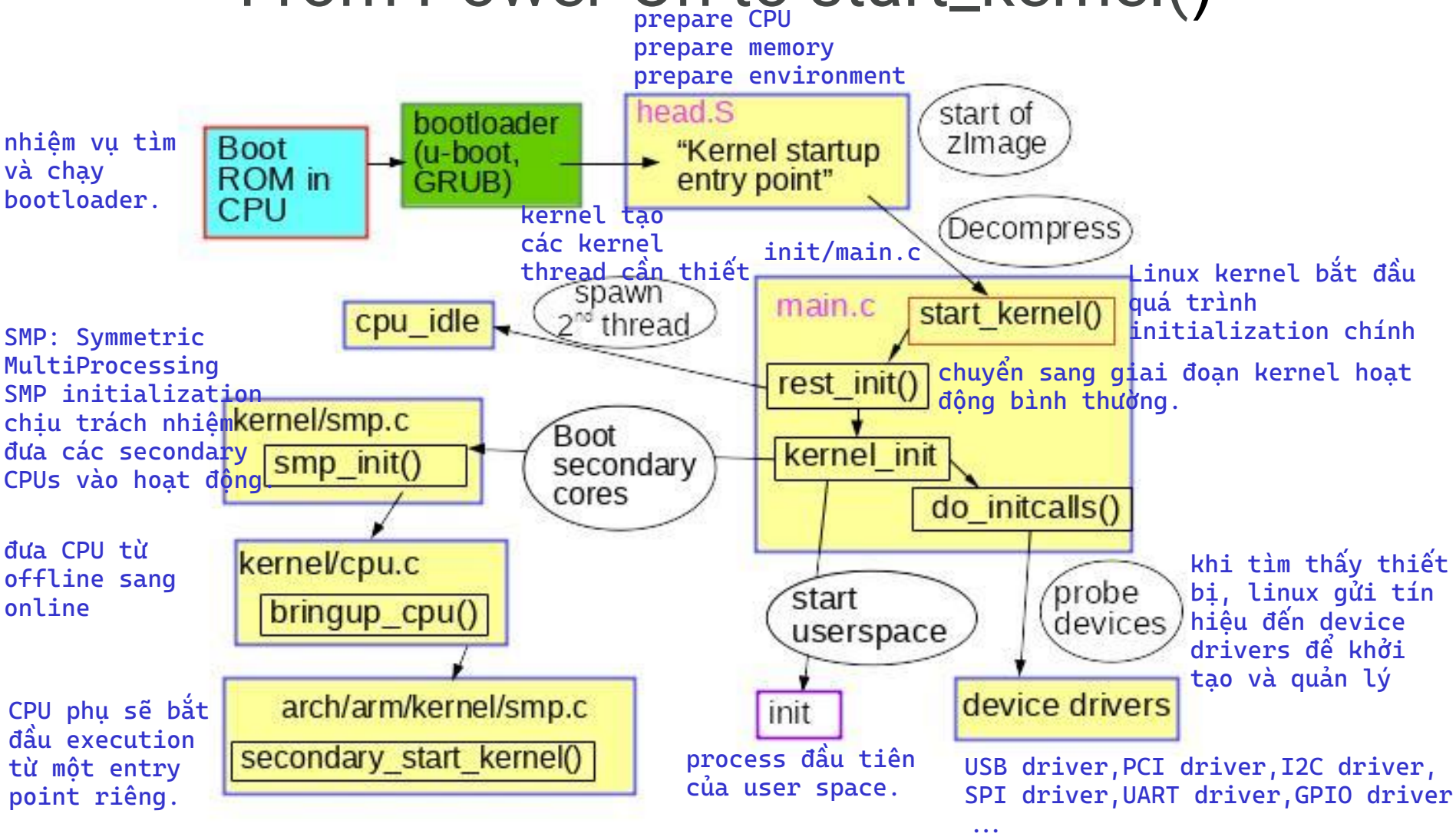
tầng 3

```
kernel entry
    ↓
architecture-specific initiali
    ↓
start_kernel()
    ↓
scheduler
memory management
interrupts
drivers
VFS
network
 ...
    ↓
userspace
```

FPT Education

FPT UNIVERSITY

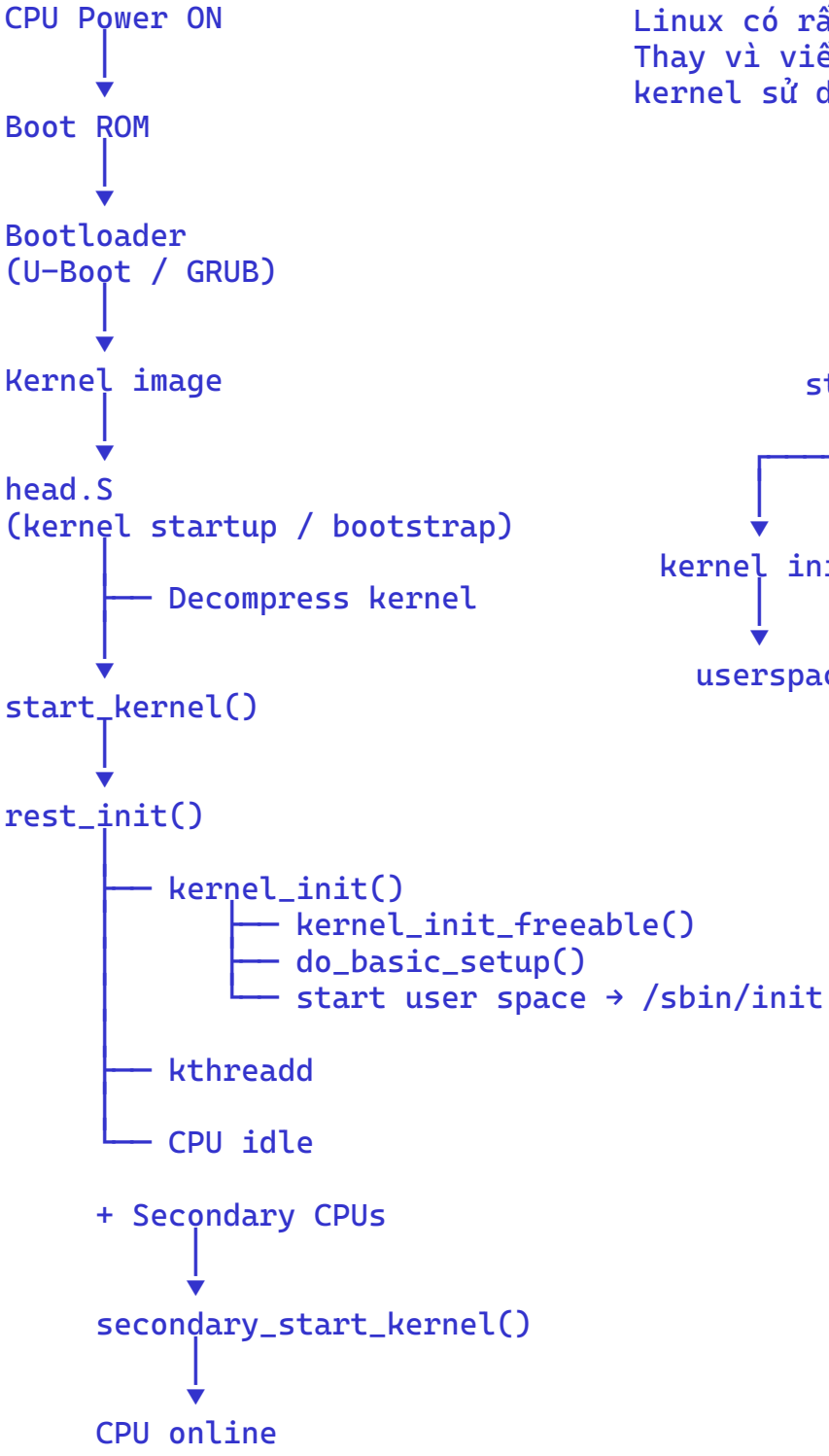
From Power-On to start_kernel()



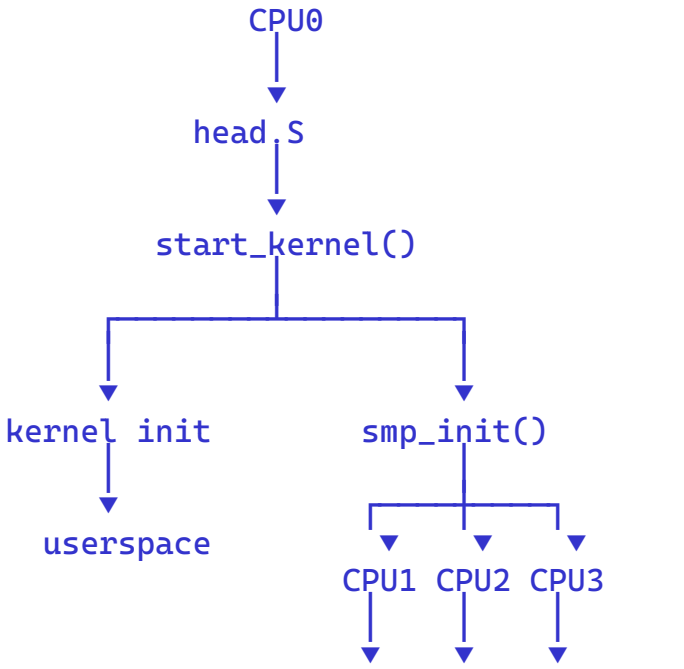
GRUB là desktop/server bootloader (x86) , U-Boot (embedded)

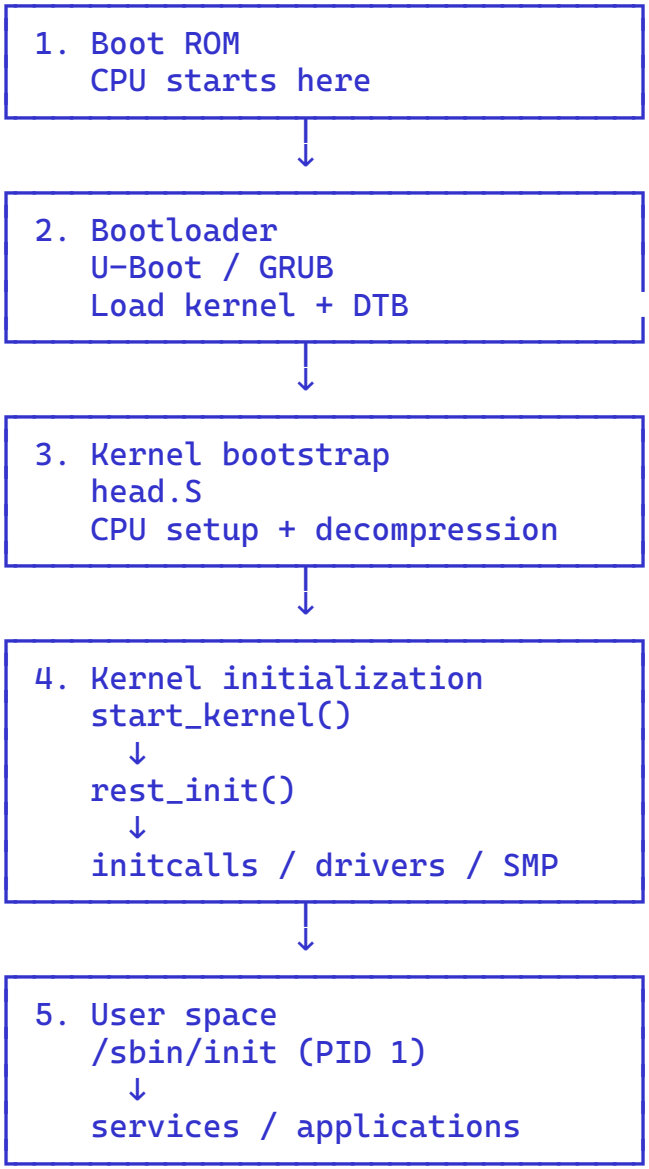
Bootloader là chương trình nằm giữa firmware/Boot ROM và kernel.

PC/x86: BIOS/UEFI ⇒ GRUB ⇒ Linux kernel Embedded ARM: Boot ROM ⇒ U-boot ⇒ Linux kernel



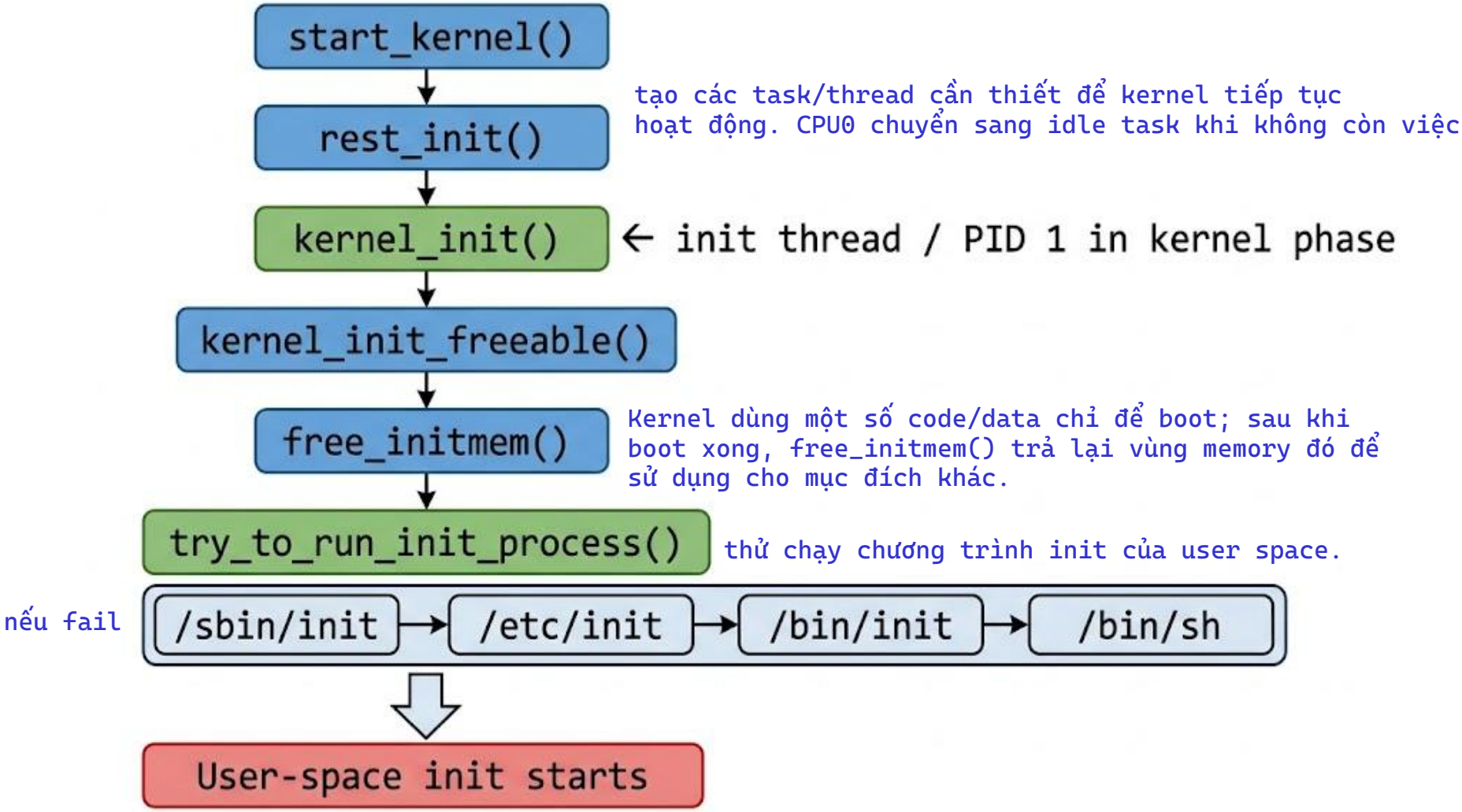
do_initcalls()
Linux có rất nhiều subsystem/driver cần initialization.
Thay vì viết tất cả initialization trực tiếp trong một hàm rất lớn,
kernel sử dụng initcall mechanism.





FPT Education
FPT UNIVERSITY

Kernel Initialization



kernel_init() là kernel thread/process có PID 1 trong giai đoạn kernel; sau khi hoàn tất kernel initialization, nó chạy chương trình user-space init, và PID 1 từ đó trở thành user-space init process.

kernel_init_freeable()
kernel initialization phần còn lại, sau đó các vùng memory chỉ cần trong quá trình initialization có thể được giải phóng.

Thành phần	Ý nghĩa
`start_kernel()`	Bắt đầu kernel initialization chính
`rest_init()`	Tạo các task/thread quan trọng và chuyển sang giai đoạn tiếp theo
`kernel_init()`	Code chạy bởi PID 1 trong **kernel phase**
`kernel_init_freeable()`	Hoàn thành các initialization còn lại
`free_initmem()`	Giải phóng memory chỉ dùng trong quá trình boot
`try_to_run_init_process()`	Tìm và chạy chương trình init của user space
`/sbin/init`	Init program thường được ưu tiên
`/etc/init`	Fallback init path
`/bin/init`	Fallback init path
`/bin/sh`	Fallback cuối cùng, có thể dùng shell làm PID 1
`execve()`	Thay thế chương trình đang chạy bằng init program
`PID 1`	Process đầu tiên của user space, có vai trò đặc biệt
User-space init	Bắt đầu giai đoạn user space

FPT Education
FPT UNIVERSITY

Flow of Control

- Bootloader → Bootstrap → head.o → start_kernel()
- head.o: CPU setup, MMU enable
- Before MMU: physical addressing
- After MMU: virtual memory

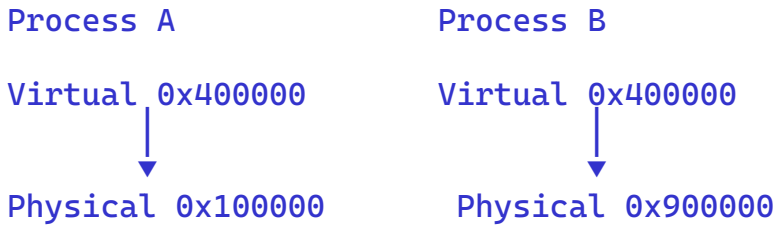
Memory Management Unit

chịu trách nhiệm chính cho việc chuyển đổi:

Virtual Address
↓
MMU
↓
Physical Address

Trước khi MMU translation được bật, CPU không thực hiện virtual-to-physical address translation theo page tables.

Sau khi MMU được enable:



FPT Education
FPT UNIVERSITY

hàm khởi tạo chính của Linux kernel.

Kernel Startup: start_kernel()

- Entry point in init/main.c
- Initializes memory, scheduler, interrupts
- Calls setup_arch() hàm để thực hiện các bước khởi tạo phụ thuộc vào kiến trúc CPU/platform.
- Never returns - it transitions the system into a running OS.

Trước start_kernel():
CPU vừa boot
↓
Kernel code được nạp
↓
CPU setup
MMU
memory
...
Hệ thống chưa thực sự trở thành một OS hoàn chỉnh.

Sau quá trình kernel initialization:

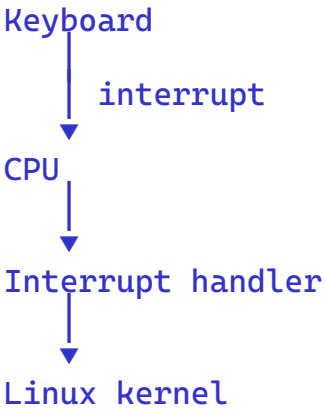
Memory management ✓
Scheduler ✓
Interrupts ✓
Timers ✓
Devices ✓
Kernel threads ✓
Root filesystem ✓
↓
User space
↓
init / systemd
↓
Running Linux OS

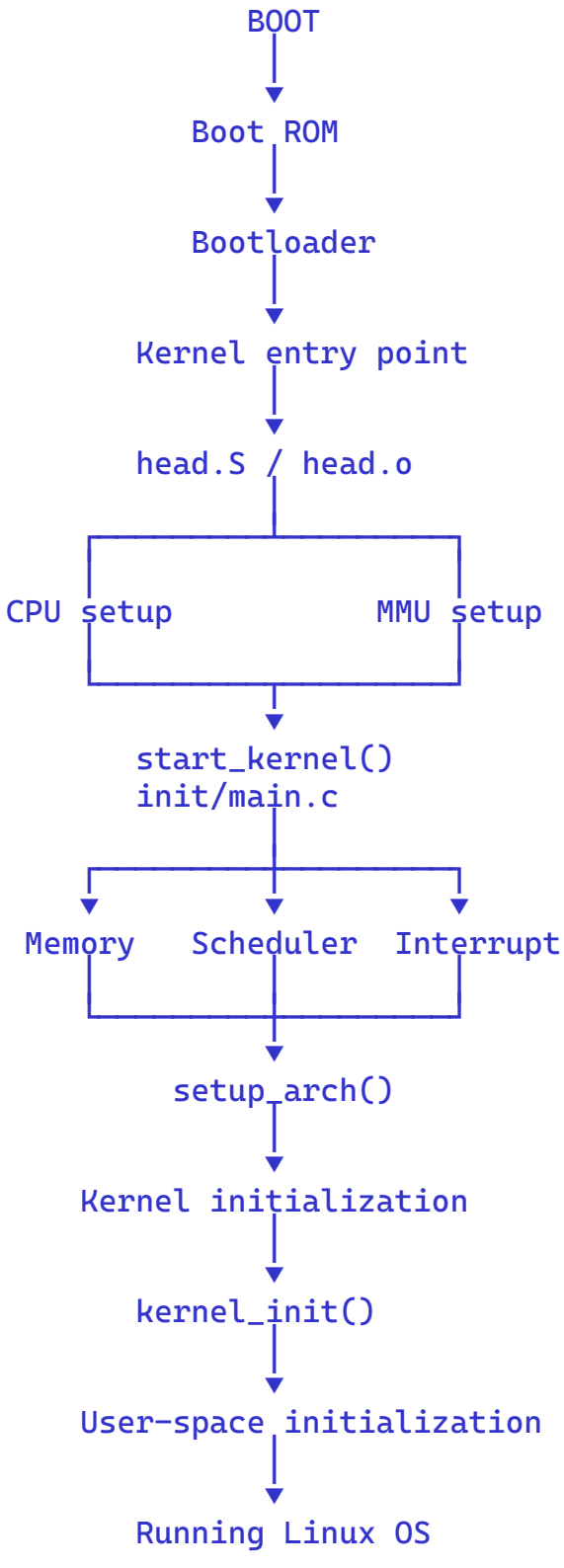
Scheduler initialization



A: chạy
B: ready
C: ready

Interrupt initialization





head.o = chuẩn bị CPU

start_kernel() = khởi tạo Linux kernel

kernel_init() = đưa kernel sang user space

FPT Education
FPT UNIVERSITY

Kernel Command Line

chuỗi tham số mà bootloader truyền cho Linux kernel để kernel biết phải khởi động như thế nào.

Console chính của kernel nằm ở thiết bị ttyS0 Root filesystem nằm trên thiết bị này.

- Example: console=ttyS0 root=/dev/mmcblk0
- __setup macro binds parameters to functions
- Controls early boot behavior

Macro __setup liên kết một tên/tham số trên command line với một hàm xử lý tham số đó.

VD: __setup("foo=", foo_setup); Kernel command line: foo=123

thì kernel sẽ tìm handler tương ứng: foo=123 ⇒ "foo=" ⇒ foo_setup() ⇒ xử lý giá trị 123

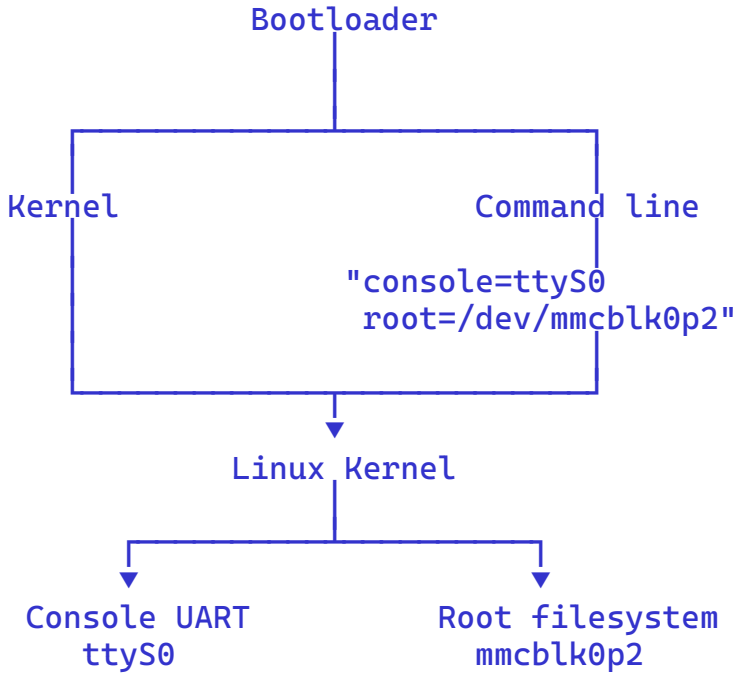
Param	Meaning
console	Output device
root	Root filesystem
init	First process

VD: console=ttyS0 kernel log ra UART.

root=/dev/mmcblk0p2 root filesystem nằm trên partition 2 của SD/eMMC.

mem=512M giới hạn lượng RAM kernel sử dụng.

init=/bin/myinit sau khi kernel initialization hoàn tất, kernel cố gắng chạy /bin/myinit thay


Bootloader
Kernel
Command line
"console=ttyS0
root=/dev/mmcblk0p2"
Linux Kernel
Console UART
ttyS0
Root filesystem
mmcblk0p2

FPT Education
FPT UNIVERSITY

Subsystem Initialization

giai đoạn Linux bắt đầu lần lượt khởi tạo các subsystem như memory management, filesystem, driver, network, device ... sau khi đã vào start_kernel().

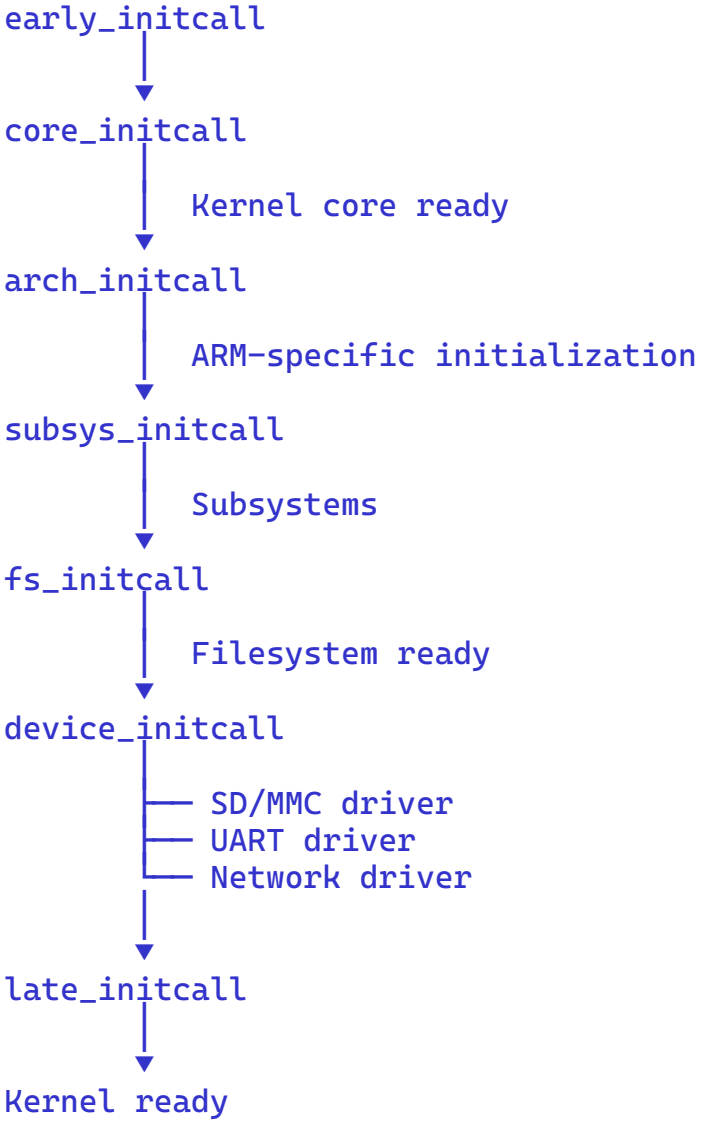
Một hàm initialization được kernel gọi trong quá trình boot.

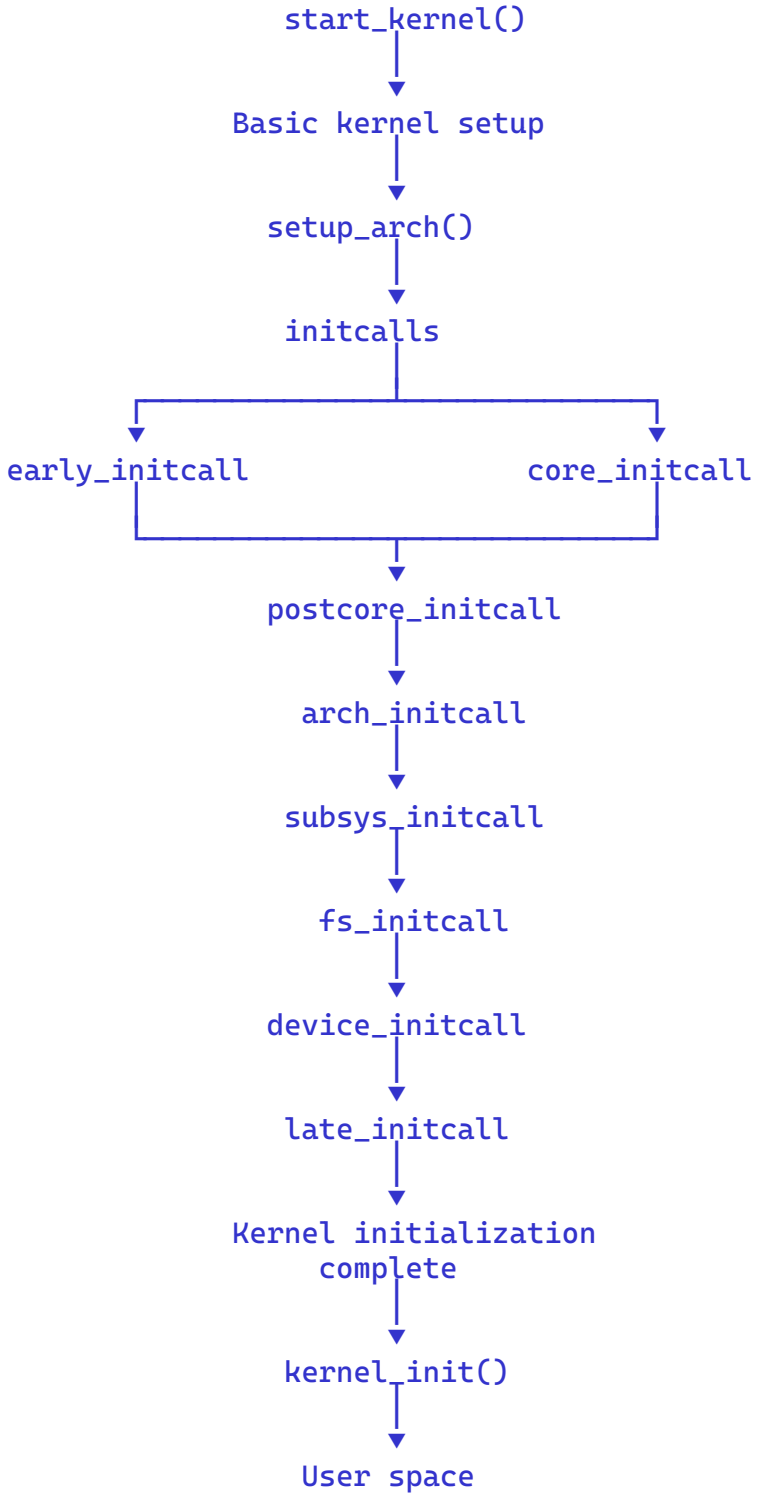
- Initcall levels control order: define boot ordering without hardcoding dependencies.
  - early_initcall
  - core_initcall
  - postcore_initcall
  - arch_initcall
  - subsys_initcall
  - fs_initcall
  - device_initcall
  - late_initcall

EARLY (Sớm)
↓
CORE
↓
POSTCORE
↓
ARCH
↓
SUBSYS
↓
FS
↓
DEVICE
↓
LATE (muộn)

- early → core → device → late
- Stored in .init.text
- Freed after boot

Chứa code chỉ cần trong quá trình initialization.
.text : code cần tồn tại trong thời gian kernel chạy


early_initcall
core_initcall
Kernel core ready
arch_initcall
ARM-specific initialization
subsys_initcall
Subsystems
fs_initcall
Filesystem ready
device_initcall
SD/MMC driver
UART driver
Network driver
late_initcall
Kernel ready


start_kernel()
Basic kernel setup
setup_arch()
initcalls
early_initcall
core_initcall
postcore_initcall
arch_initcall
subsys_initcall
fs_initcall
device_initcall
late_initcall
Kernel initialization complete
kernel_init()
User space

giai đoạn cuối của quá trình kernel khởi động

kernel tạo ra các process cần thiết, sau đó khởi chạy process đầu tiên của user space

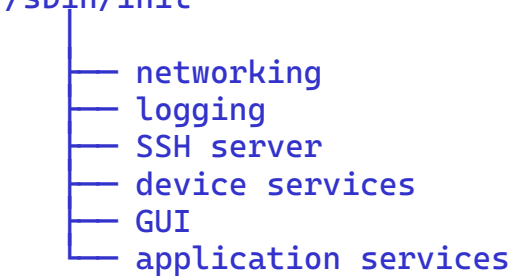
Spawning User Space

Kernel cần một task để CPU thực thi ngay từ đầu. Trong quá trình boot, task này thực hiện các công việc khởi tạo kernel.
Sau khi kernel tạo được các task cần thiết, vai trò chính của PID 0 trên CPU sẽ trở thành idle task.

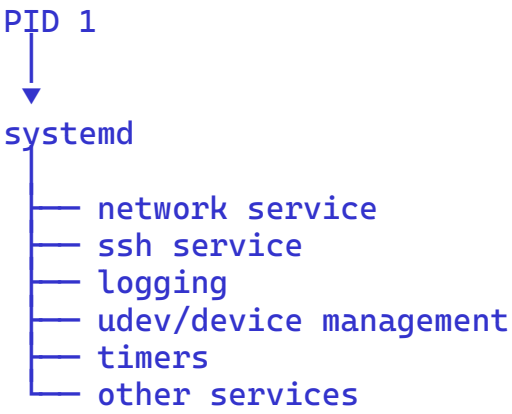
- The kernel starts with the initial boot thread (PID 0).
- After creating kernel_init (PID 1) and kthreadd (PID 2), the original boot thread becomes the idle thread for CPU 0.
- PID 0 runs the idle loop when no runnable tasks exist.
- Additional CPUs later create their own idle threads during SMP initialization.
- Executes /sbin/init (often systemd)
  - systemd manages all services, daemons, and dependencies.
  - Embedded Linux often uses lightweight alternatives:
    - BusyBox init, runit, s6, OpenRC
    - custom shell init scripts
- Kernel becomes idle process
- System ready for user space

Sau khi hoàn tất quá trình khởi tạo và chuyển sang user space, task khởi đầu của kernel trên mỗi CPU đảm nhiệm vai trò idle task khi CPU không có task runnable.

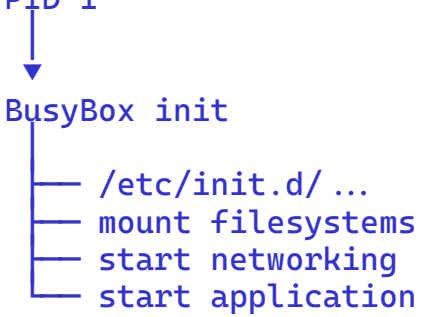
PID 1 chạy /sbin/init
/sbin/init là process đầu tiên của user space.
có nhiệm vụ khởi động phần còn lại của hệ thống.

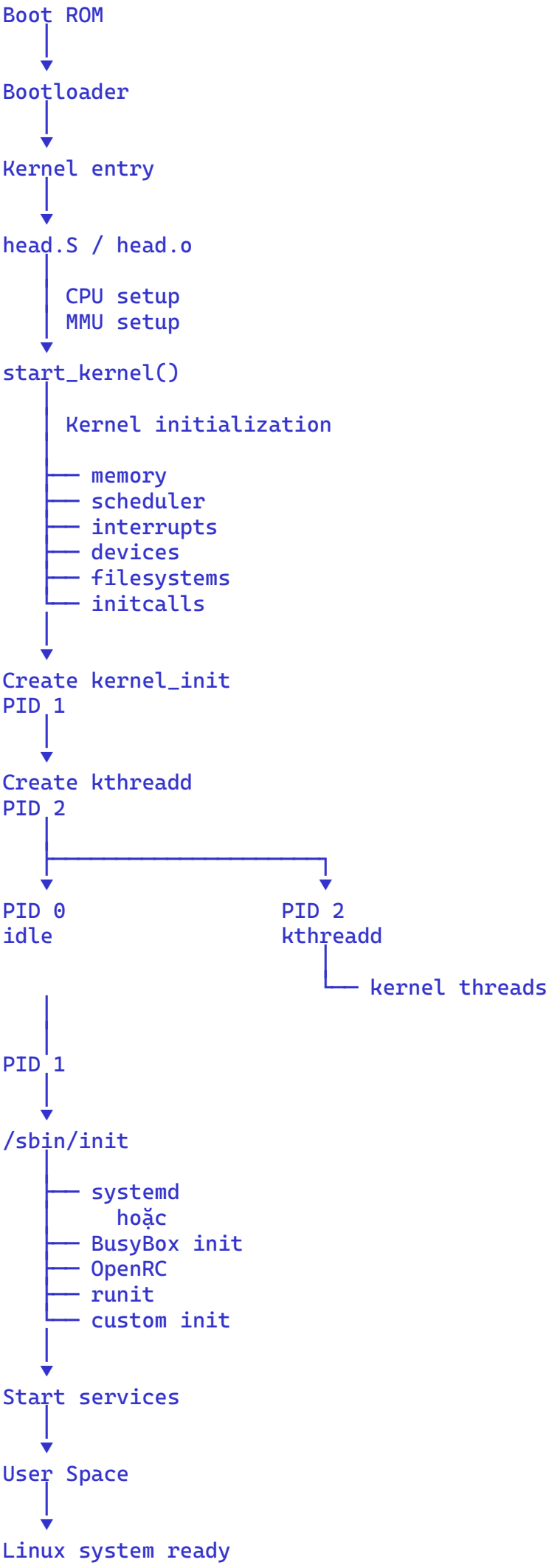


Trên các Linux desktop/server hiện đại,
/sbin/init thường dẫn tới: systemd



Embedded Linux thường không dùng systemd
có thể dùng BusyBox init



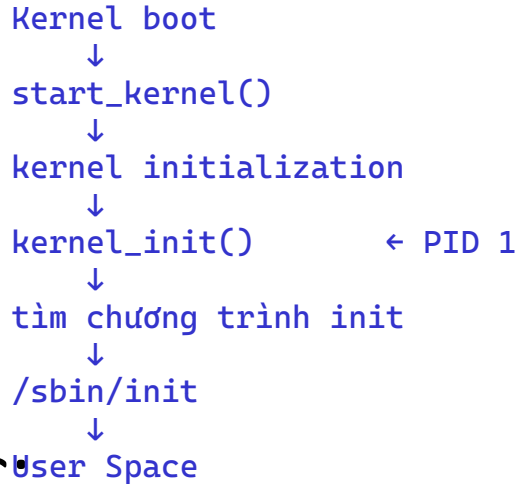

Boot ROM
Bootloader
Kernel entry
head.S / head.o
CPU setup
MMU setup
start_kernel()
Kernel initialization
memory
scheduler
interrupts
devices
filesystems
initcalls
Create kernel_init
PID 1
Create kthreadd
PID 2
PID 0
idle
PID 2
kthreadd
kernel threads
PID 1
/sbin/init
systemd
hoặc
BusyBox init
OpenRC
runit
custom init
Start services
User Space
Linux system ready

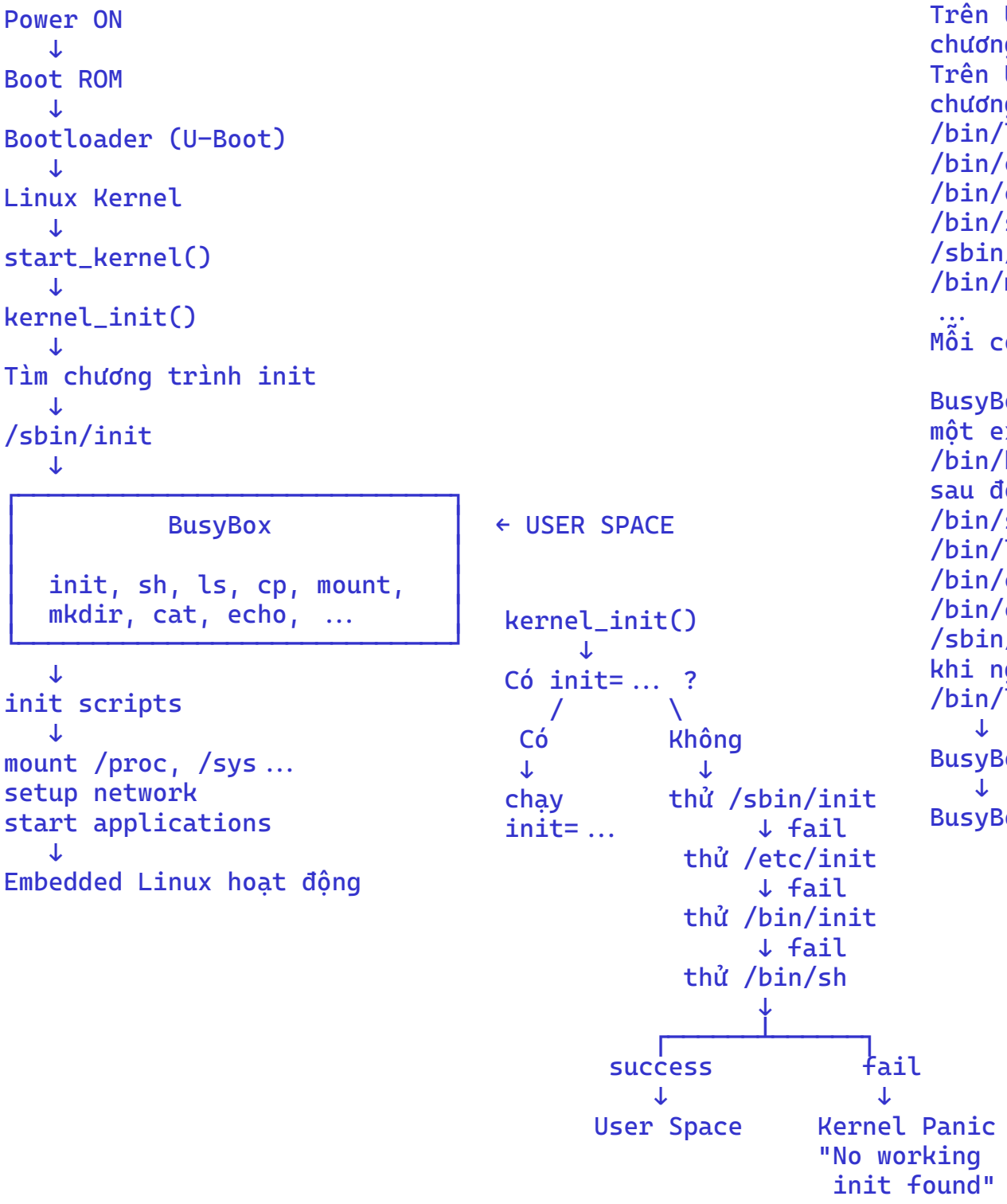
FPT Education
FPT UNIVERSITY

Init Process Startup

chuyển từ kernel space sang chạy chương trình đầu tiên trong user space

- The kernel launches the first user-space process from kernel_init().
- The init= kernel parameter has highest priority.
- If not specified, the kernel searches in order:
  - /sbin/init nếu không chạy được
  - /etc/init
  - /bin/init
  - /bin/sh shell
- If all fail, the kernel triggers a panic:
  "No working init found"





Trên Ubuntu thông thường, bạn có rất nhiều chương trình riêng:

Trên Ubuntu thông thường, bạn có rất nhiều chương trình riêng:

/bin/ls
/bin/cp
/bin/cat
/bin/sh
/sbin/init
/bin/mkdir
...

Mỗi command có thể là một executable riêng.

BusyBox gom rất nhiều command Linux/Unix cơ bản vào một executable duy nhất:
/bin/busybox
sau đó tạo các liên kết:

/bin/sh → busybox
/bin/ls → busybox
/bin/cp → busybox
/bin/cat → busybox
/sbin/init → busybox

khi người dùng chạy ls, thực chất là:

/bin/ls
↓
BusyBox
↓
BusyBox thực hiện chức năng ls

Summary

- **Kernel Structure:** Understanding the complex structure and composition of the final kernel image is key to customizing your own embedded projects.
- **Bootstrap Loader:** Many architectures concatenate a specific bootstrap loader to the kernel binary to properly set up the execution environment.
- **Flow of Control:** The boot flow moves from the kernel entry point in head.o into the primary C initialization logic within main.c.
- **Command-Line Processing:** The kernel uses advanced coding techniques, such as linker-produced tables, to declare and process command-line parameters and setup routines.
- **User Space Transition:** The final kernel boot steps produce the first user space processes, which is crucial for customizing or troubleshooting embedded startup behaviors.